

Literature Survey

A Text Based Data Extraction Pipeline for ML-Ready Datasets

Project: LoomSet

Soham Gurav

August 16, 2026

Abstract

Niche topics often lack ample open source, ML ready datasets, and manually assembling such datasets from the web is slow and time consuming. This project aims to connect data scraping and data pre processing into a single, open source, zero monetary cost pipeline. This literature survey reviews the prior work across the pipeline's three core concerns : web scraping, data cleaning, and data labeling, (in ascending order of publication year within each category). Each entry lists the paper's publication year, authors, summary, and a limitation verified by studying the paper. A summary table at the end consolidates every reviewed paper with its category, year, limitation, and a strategy for overcoming that limitation within the LoomSet architecture.

1 Web Scraping

This section reviews prior work on automated web data extraction:

1.1 DEiXTo: A Web Data Extraction Suite

Kokkoras, F., Ntonas, K., & Bassiliades, N. (2013). DEiXTo: A web data extraction suite. Proceedings of the 6th Balkan Conference in Informatics (BCI '13), ACM, 9–12. <https://dl.acm.org/doi/10.1145/2490257.2490297>

Summary: This paper introduces DEiXTo, it is a DOM based extraction suite composed of three parts: GUI-DEiXTo for building extraction rules, DEiXToBot (a Perl based Mechanize browsing agent), and DEiXTo CLE, a command line executor that applies one rule across many pages and writes structured output, including directly into a relational database.

Limitation: Verified this on the publisher's page and the paper itself: DEiXTo "consists of both freeware and open source components," i.e. it is only partially open source, which conflicts with the idea of a zero cost, fully open pipeline. Extraction rules are also authored per site template and do not adapt automatically to structural changes.

1.2 RCrawler: An R Package for Parallel Web Crawling and Scraping

Khalil, S., & Fakir, M. (2017). *RCrawler: An R package for parallel web crawling and scraping*. SoftwareX, 6, 98–106. <https://doi.org/10.1016/j.softx.2017.04.004>

Summary: This paper presents RCrawler, the first parallel domain based web crawler implemented in the R environment, contributed as an R package for content scraping alongside crawling.

Limitation: In the paper’s overview of R toolkits: existing R packages such as rvest, tm.plugin.webmining, and scrapeR provide no built in crawling capability, they can only parse and extract content from URLs that the user has already collected and supplied manually.

1.3 Web Scraping: Applications and Scraping Tools

Matta, P., Sharma, N., Sharma, D., Pant, B., & Sharma, S. (2020). *Web scraping: Applications and scraping tools*. International Journal of Advanced Trends in Computer Science and Engineering, 9(1). <https://doi.org/10.30534/ijatcse/2020/185952020>

Summary: This paper surveys the landscape of web scraping applications and compares some commonly used scraping tools like FMiner, Octoparse, ParseHub, Web Content Extractor, Scrapy, DataOx, Scraping-Bot.io, and Webhose.io

Limitation: From the paper: the paper is a descriptive/comparative survey rather than a working system. Among the tools it favors, FMiner is explicitly noted as not free for use and Scrapy requires the user to manually select or build a field template before scraping, hence tool surveyed is both fully automated or free.

1.4 CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data

Wenzek, G., Lachaux, M.-A., Conneau, A., Chaudhary, V., Guzmán, F., Joulin, A., & Grave, É. (2020). *CC-Net: Extracting high quality monolingual datasets from web crawl data*. Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020), 4003–4012. <https://doi.org/10.48550/arXiv.1911.00359>

Summary: This paper describes a pipeline for filtering and deduplicating common crawl snapshots into high quality, per language monolingual corpora using perplexity based quality filtering against wikipedia language models.

Limitation: As noted in the paper, CCNet is built for large, generic web crawl corpora rather than targeted, niche topic scraping because it just filters an already crawled dump for language quality but has no mechanism for discovering or prioritizing pages relevant to a specific niche subject.

1.5 A Framework for Automated Scraping of Structured Data Records From the Deep Web Using Semantic Labeling: Semantic Scraper

Kumaresan, U., & Ramanujam, K. (2022). *A framework for automated scraping of structured data records from the deep web using semantic labeling: Semantic Scraper*. International Journal of Information Retrieval Research, 12(1), 1–18. <https://doi.org/10.4018/IJIRR.290830>

Summary: This paper proposes Semantic Scraper, which identifies data rich regions of a page by scoring leaf nodes for informative content repetition (rather than syntactic/template structure) and achieves 100% precision in locating data rich regions across 31 real world datasets spanning 7 domains.

Limitation: Verified in the paper’s own discussion: computational complexity is $O(n^2)$ due to similarity computation across the Semantic Feature Tree; accuracy degrades when only a single source page is available, when attributes are missing, or when the same template is reused inconsistently for different attributes.

1.6 Web Scraping for Data Analytics: A BeautifulSoup Implementation

(2023). *Web scraping for data analytics: A BeautifulSoup implementation*. IEEE Conference Publication (IEEE Xplore document 10145369). <https://doi.org/10.1109/WiDS-PSU57071.2023.00025>

Summary: This paper builds a BeautifulSoup based scraper demonstrated on Amazon product pages (name, price, review count, rating, link) and pairs it with a Matplotlib based visualization interface to analyze the scraped results.

Limitation: Confirmed in the abstract: the authors state the limitations of the implementation “mainly revolved around applying it to specific product names rather than generic ones,” meaning the extraction logic is tuned to one site/category and does not generalize automatically to arbitrary page structures.

1.7 Web Scraping: Huge Data Collection from Web

Bhatt, C., et al. (2023). *Web scraping: Huge data collection from web*. 2023 International Conference on Sustainable Emerging Innovations in Engineering and Technology (ICSEIET), IEEE. <https://doi.org/10.1109/ICSEIET58677.2023.10303037>

Summary: This paper reviews large scale web scraping approaches for data collection, framing scraping as part of a broader technical communication and analytics pipeline, with attention to the ethical dimension of collecting data at scale.

Limitation: As referenced in the the paper’s framing, full text access is limited. the identifiable gap is that responsible scraping practice (respecting a site’s own regulations at scale) is discussed only at a conceptual level rather than implemented as an enforceable pipeline control.

1.8 Software Architecture for Improving Scraping Systems Using Artificial Intelligence

Posedaru, B.-Ş., Bătăgan, L., Bologa, R., Plăcintă, D.-D., & Mirea, C.-M. (2024). *Software architecture for improving scraping systems using artificial intelligence*. Economic Cybernetics Studies and Research, 58(3). <https://doi.org/10.24818/18423264/58.3.24.09>

Summary: This paper proposes an autonomous scraper that integrates NLP and ChatGPT to identify CSS selectors from a page’s HTML and autonomously generate scraper code from a natural language description of the target data.

Limitation: Confirmed in the paper: it relies on ChatGPT (a proprietary, paid API) to determine selectors, and the authors explicitly list HTML clean up and DOM parsing improvements as future work rather than

solved problems conflicting with a zero monetary cost, complete pipeline.

1.9 Scalable and Adaptive Web Scraping Framework for Extracting Diverse Data from Open Internet Sources

(2024). *Scalable and adaptive web scraping framework for extracting diverse data from open internet sources*. 2024 International Conference on Power, Communication, and Signal Processing Networks (ICPCSN), IEEE. <https://doi.org/10.1109/ICPCSN62568.2024.00147>

Summary: This paper describes a system combining headless browsing, API integration, and HTML parsing to dynamically adjust to different websites’ structures and data formats for broad, multi purpose data collection.

Limitation: Confirmed from the abstract: the framework is presented at a conceptual/architectural level : it discusses scalability and adaptability as goals the design pursues, but the abstract provides no benchmarked throughput or scale figures to substantiate the “scalable” claim.

1.10 Automating Web Data Collection: Challenges, Solutions, and Python-Based Strategies for Effective Web Scraping

Wahed, M. A., & Bader, A. F. (2024). *Automating web data collection: Challenges, solutions, and Python based strategies for effective web scraping*. Proceedings of the 7th International Conference on Internet Applications, Protocols and Services (NETAPPS 2024), Kuala Lumpur. <https://doi.org/10.1109/NETAPPS63333.2024.10823528>

Summary: This paper classifies the major challenges of automated web-data collection (changing HTML structure, access barriers, ethical/legal concerns) and maps each to Python library based solutions and mitigation strategies.

Limitation: Confirmed in the paper’s own framing: it acknowledges that scraping is “causing controversy due to ethical concerns” and that many sites’ terms of service explicitly forbid automated scraping; the paper classifies this risk but does not resolve it technically; solutions remain library specific rather than systemic.

1.11 LLMs Applied to Web Scraping and Web Crawling: A Systematic Review

(2026). *LLMs applied to web scraping and web crawling: a systematic review*. Computing. <https://doi.org/10.1007/s00607-026-01666-5>

Summary: Synthesizes evidence across studies using LLMs (BERT-family, GPT-family, LLaMA, Mistral, Gemini) for scraping and crawling automation, reporting evaluation practices (F1, BLEU, RAGAS) and a trend toward Small Language Models (SLMs) for resource constrained, domain specific tasks.

Limitation: As synthesized in the source notebook from the review itself: HTML complexity, computational cost, token limits, data bias, and legal risk remain open challenges across the surveyed literature. Being a review rather than a new system, it does not itself resolve any of these, it aggregates them as future work (SLM adoption, hybrid pipelines, standardized benchmarks).

2 Data Cleaning

This section reviews prior work on automating the detection and repair of data quality problems, ordered by ascending publication year.

2.1 A Model and System for Querying Provenance from Data Cleaning Workflows

(2021). *A model and system for querying provenance from data cleaning workflows*. In *Advances in Databases and Information Systems (ADBIS 2021)*, Springer, LNCS. https://doi.org/10.1007/978-3-030-80960-7_11

Summary: This paper proposes a provenance model and querying system that tracks how records are transformed through the stages of a data cleaning workflow, enabling users to trace which cleaning step produced a given output value.

Limitation: Based on the available bibliographic description, the system addresses provenance querying and explaining what a cleaning pipeline did rather than performing error detection or repair itself, so it must be paired with an actual cleaning engine to be useful end to end.

2.2 Auto-Prep: Efficient and Automated Data Preprocessing Pipeline

Bilal, M., Ali, G., Iqbal, M. W., Anwar, M., Malik, M. S. A., & Abdul Kadir, R. (2022). *Auto-Prep: Efficient and automated data preprocessing pipeline*. IEEE Access, 10, 107764–107784. <https://doi.org/10.1109/ACCESS.2022.3198662>

Summary: This paper is about developing a Python based auto preprocessing architecture that detects data problems, visualizes them for the user, and recommends the most effective cleaning/preparation method after evaluating candidate techniques, evaluated on ten diverse datasets.

Limitation: Confirmed in the abstract: the system’s value depends on evaluating multiple candidate transformation techniques before recommending one, adding computation overhead versus a directly chosen pipeline; validation is limited to ten datasets, which constrains claims of general applicability.

2.3 Data Cleaning and Machine Learning: A Systematic Literature Review

Côté, P.-O., Nikanjam, A., Ahmed, N., Humeniuk, D., & Khomh, F. (2023). *Data cleaning and machine learning: A systematic literature review*. arXiv:2310.01765 (also Automated Software Engineering, 2024). <https://doi.org/10.1007/s10515-024-00453-w>

Summary: This paper systematically reviews 2016 to 2022 literature on the dual relationship between data cleaning and ML, covering feature cleaning, label cleaning, entity matching, outlier detection, imputation, and holistic cleaning, and provides future work recommendations.

Limitation: Confirmed in the abstract and body: as a review rather than a tool, it explicitly reports that many surveyed techniques “are promising but still need extension and future work,” i.e. it maps and diagnoses gaps in the field rather than closing them.

2.4 IterClean: An Iterative Data Cleaning Framework with Large Language Models

Ni, W., Zhang, K., Miao, X., Zhao, X., Wu, Y., & Yin, J. (2024). *IterClean: An iterative data cleaning framework with large language models*. Proceedings of the ACM Turing Award Celebration Conference – China 2024, 100–105. <https://doi.org/10.1145/3674399.3674436>

Summary: This paper introduces an iterative cleaning loop of error detection, verification, and repair that uses a small labeled seed (as few as 5 tuples) plus batch LLM prompting to substantially raise F1 over prior state of the art cleaning methods.

Limitation: Confirmed in the paper’s own pipeline description: a human still performs tuple selection and labeling at each iteration (human in the loop), and repeated batch prompting through an LLM means cost and latency scale with the number of cleaning iterations required.

2.5 Trends and Challenges in Data Cleaning for Large-Scale Systems

(2025). *Trends and challenges in data cleaning for large-scale systems*. International Journal of Geoinformatics/Geographic Information Systems (IJGIS). <https://doi.org/10.63412/53kcztv76>

Summary: This paper is a survey of recent (post 2020) approaches to cleaning data at scale, covering how heterogeneity, real time processing demands, and resource constraints reshape classical data cleaning techniques.

Limitation: Confirmed in the abstract: traditional cleaning techniques struggle with heterogeneous, real time, resource constrained settings, and the survey explicitly flags a lack of standardized benchmarks and unresolved scalability bottlenecks as open problems.

3 Labeling Extracted Data

This section reviews prior work on automating or semi-automating the labeling of extracted data for supervised learning, ordered by ascending publication year.

3.1 Data Programming: Creating Large Training Sets, Quickly

Ratner, A., De Sa, C., Wu, S., Selsam, D., & Ré, C. (2016). *Data programming: Creating large training sets, quickly*. Advances in Neural Information Processing Systems (NeurIPS 2016), 29, 3567–3575. <https://doi.org/10.48550/arXiv.1605.07723>

Summary: This paper introduces “data programming,” a paradigm in which users write labeling functions that encode weak-supervision heuristics or domain rules; a generative model then combines and denoises these functions’ outputs into probabilistic training labels.

Limitation: Confirmed in the paper: the approach depends on users authoring effective labeling functions, which requires domain expertise, and noisy or conflicting heuristics directly translate into label noise in the resulting training set.

3.2 Snorkel: Rapid Training Data Creation with Weak Supervision

Ratner, A., Bach, S. H., Ehrenberg, H., Fries, J., Wu, S., & Ré, C. (2017). *Snorkel: Rapid training data creation with weak supervision*. arXiv:1711.10160 (also Proceedings of the VLDB Endowment, 2018). <https://doi.org/10.14778/3157794.3157797>

Summary: Builds a full weak supervision system around the data programming paradigm, letting users write, combine, and iterate on labeling functions in place of large-scale manual labeling, with a generative model automatically estimating each function’s accuracy.

Limitation: Confirmed in the paper abstract: Snorkel still needs heuristic design expertise from its users, and downstream label quality is bounded by the quality and coverage of the labeling functions supplied.

3.3 Snuba: Automating Weak Supervision to Label Training Data

Varma, P., & Ré, C. (2018). *Snuba: Automating weak supervision to label training data*. Proceedings of the VLDB Endowment, 12(3), 223–236. <https://doi.org/10.14778/3184470.3184475>

Summary: This paper is about the generation of weak supervision heuristics themselves (rather than requiring a human to hand write every labeling function), reducing the manual rule writing burden for low resource labeling settings.

Limitation: Confirmed in the paper study : Snuba still requires a small hand-labeled seed set to bootstrap and validate the heuristics it generates, so it reduces but does not eliminate the need for initial human labeled data.

3.4 Semi-Automated Data Labeling

Desmond, M., Duesterwald, E., Brimijoin, K., Brachman, M., & Pan, Q. (2021). *Semi-automated data labeling*. Proceedings of the NeurIPS 2020 Competition and Demonstration Track, PMLR 133, 156–169. <https://doi.org/10.48550/arXiv.2012.14702>

Summary: This paper describes a system where a predictive model observes a human labeler’s decisions, recommends labels, and automates basic labeling interface functions; agreement between labeler and model is tracked via checkpoints where the human can delegate the rest of the task to the model.

Limitation: Confirmed in the abstract: the system is explicitly human-machine collaborative, a human labeler remains in the loop throughout, and the quality of the model’s recommendations depends on the labeling decisions it has already observed, so early stage recommendations are only as good as the human examples seen so far.

3.5 Auto-labelling Entities in Low-Resource Text: A Geological Case Study

Enkhsaikhan, M., Liu, W., Holden, E. J., & Duuring, P. (2021). *Auto-labelling entities in low-resource text: a geological case study*. Knowledge and Information Systems, 63(3), 695–715. <https://doi.org/10.1007/s10115-020-01532-6>

Summary: This paper proposes an iterative deep learning NER framework using distant supervision to auto label domain specific text, applied to mineral exploration reports to produce the OzROCK dataset across six

geological entity categories.

Limitation: Confirmed in the paper’s abstract: technical jargon and domain specific writing conventions make transfer learning difficult even though the surface language is English, and the authors note their approach identifies a subset of instances that remain “challenging for automatic labelling” and require attention from human experts.

3.6 Multistep Automated Data Labelling Procedure (MADLaP) for Thyroid Nodules on Ultrasound

Zhang, J., et al. (2023). *Multistep Automated Data Labelling Procedure (MADLaP) for thyroid nodules on ultrasound: An artificial intelligence approach for automating image annotation*. *Artificial Intelligence in Medicine*, 141, 102553. (Preprint: arXiv:2206.14305, 2022). <https://doi.org/10.1016/j.artmed.2023.102553>

Summary: This paper’s authors develop MADLaP, a multi module pipeline combining rule based NLP, deep learning image segmentation, and OCR to automatically identify ultrasound images of a specific thyroid nodule and assign the correct pathology label from pathology and radiology reports.

Limitation: Confirmed in the abstract: MADLaP achieved a yield of only 63% at 83% accuracy, meaning more than a third of cases could not be automatically labeled at all and still require manual annotation and it depends on having multiple structured input types (pathology reports, ultrasound images, radiology reports) simultaneously available.

4 Summary Table

Table 1 consolidates all twenty two reviewed papers with their category, publication year, principal limitation, and a proposed strategy for overcoming that limitation within LoomSet’s open-source architecture.

Table 1: Summary of reviewed literature: limitations and proposed mitigations within the LoomSet pipeline.

Paper Title	Category	Year	Limitation	How to Overcome It
DEiXTo: A Web Data Extraction Suite	Web Scraping	2013	Only partially open source (freeware + open-source components); rules do not adapt to structural change.	Replace freeware components with a fully open-source rule engine (Scrapy/Playwright) and auto-detect selector drift.
RCrawler: An R Package for Parallel Web Crawling and Scraping	Web Scraping	2017	Requires manually collected URLs; no built-in crawling/discovery.	Pair with an automated discovery stage (duckduckgo-search / SearxNG) to generate seed URLs.
Web Scraping: Applications and Scraping Tools	Web Scraping	2020	Descriptive survey; favored tools (e.g. FMiner) are paid; Scrapy needs manual templates.	Standardize on free/open tools only (Scrapy, Playwright) with scripted template selection.
CCNet: Extracting High Quality Monolingual Datasets from Web Crawl Data	Web Scraping	2020	Built for generic web-crawl corpora, not targeted niche-topic scraping.	Add a niche-topic relevance filter (embedding/keyword match) before generic quality filtering.
A Model and System for Querying Provenance from Data Cleaning Workflows	Data Cleaning	2021	Addresses provenance querying, not error detection/repair itself.	Log provenance metadata alongside the SQLite/Parquet store so lineage and cleaning happen together.
A Framework for Automated Scraping of Structured Data Records From the Deep Web Using Semantic Labeling: Semantic Scraper	Web Scraping	2022	$O(n^2)$ complexity; degrades with single-source pages, missing attributes, or inconsistent formatting.	Bound cost by sampling/pruning candidate leaf nodes; fall back to rule-based extraction for single-source pages.
Auto-Prep: Efficient and Automated Data Preprocessing Pipeline	Data Cleaning	2022	Evaluating multiple candidate techniques adds overhead; validated on only ten datasets.	Allow manual override/config for known data types to skip redundant evaluation.

Continued on next page

Table 1 – continued from previous page

Paper Title	Category	Year	Limitation	How to Overcome It
Web Scraping for Data Analytics: A BeautifulSoup Implementation	Web Scraping	2023	Tuned to one site/category (Amazon); does not generalize automatically.	Use a general-purpose content extractor (trafilatura) instead of hand-tuned, site-specific selectors.
Web Scraping: Huge Data Collection from Web	Web Scraping	2023	Responsible-scraping practice discussed only conceptually, not enforced.	Add an explicit robots.txt / ToS compliance and rate-limiting layer as a pipeline stage.
Data Cleaning and Machine Learning: A Systematic Literature Review	Data Cleaning	2023	A review, not a tool; maps gaps rather than closing them.	Prioritize the review’s well-validated techniques now; track newer literature as an ongoing update process.
Multistep Automated Data Labelling Procedure (MADLaP) for Thyroid Nodules on Ultrasound	Labeling	2023	Yield of only 63% (83% accuracy); needs multiple structured input types simultaneously.	Combine multiple weak signals and route the low-confidence/unlabeled fraction to human review.
Software Architecture for Improving Scraping Systems Using Artificial Intelligence	Web Scraping	2024	Depends on paid ChatGPT API; DOM-parsing improvements left as future work.	Swap in a local open-source LLM via Ollama and complete the DOM-parsing module in-house.
Scalable and Adaptive Web Scraping Framework for Extracting Diverse Data from Open Internet Sources	Web Scraping	2024	Conceptual/architectural only; no benchmarked scalability figures.	Benchmark throughput explicitly and scale horizontally via Scrapy’s concurrent worker pool.
Automating Web Data Collection: Challenges, Solutions, and Python-Based Strategies for Effective Web Scraping	Web Scraping	2024	Classifies legal/ethical risk but does not resolve it; solutions are library-specific.	Encode robots.txt / ToS parsing and consent checks as a mandatory pre-scrape gate.
IterClean: An Iterative Data Cleaning Framework with Large Language Models	Data Cleaning	2024	Human-in-the-loop tuple selection each iteration; cost/latency scale with LLM prompting.	Run the LLM locally via Ollama and cap iterations with an early-stopping rule.

Continued on next page

Table 1 – continued from previous page

Paper Title	Category	Year	Limitation	How to Overcome It
Trends and Challenges in Data Cleaning for Large-Scale Systems	Data Cleaning	2025	Struggles with heterogeneity, real-time demands, resource constraints; no standard benchmarks.	Add streaming/incremental cleaning and adopt standard evaluation benchmarks at the export stage.
LLMs Applied to Web Scraping and Web Crawling: A Systematic Review	Web Scraping	2026	Aggregates open challenges (HTML complexity, cost, token limits, bias, legal risk) without resolving them.	Adopt small local models (Ollama) to cut cost, and route low-confidence extractions to human review.
Data Programming: Creating Large Training Sets, Quickly	Labeling	2016	Requires domain expertise to write labeling functions; noisy heuristics propagate as label noise.	Provide a library of reusable labeling-function templates per niche domain.
Snorkel: Rapid Training Data Creation with Weak Supervision	Labeling	2017	Needs heuristic-design expertise; label quality bounded by function quality/coverage.	Ship default labeling-function templates plus the generative denoising model as a starting baseline.
Snuba: Automating Weak Supervision to Label Training Data	Labeling	2018	Still requires a small hand-labeled seed set to bootstrap heuristics.	Bootstrap the seed set using LoomSet’s own human-in-the-loop review stage (Label Studio).
Semi-Automated Data Labeling	Labeling	2021	Human labeler remains in the loop throughout; recommendations bounded by examples seen so far.	Route only low-confidence predictions to the human-in-the-loop review stage (Label Studio).
Auto-labelling Entities in Low-Resource Text: A Geological Case Study	Labeling	2021	Domain jargon hampers transfer learning; some instances remain unlabelable automatically.	Build small domain-specific dictionaries/rules per niche topic and escalate uncertain spans to human review.